Design and Implementation of AI-Enabled Swarm Robots for Autonomous Fire Detection and Suppression

Shuvo Ghose Shishir (2019338045)
Md. Rimon Hossain (2020338007)
Saeed Ahamed Mridha (2020338012)
Md. Rashedul Islam (2020338023)
Md. Sohel Rana (2020338034)
Md. Mehedi Hasan (2020338037)
Kowshic Mazumder Niloy (2019338079)

July 24, 2025

Microprocessor and Interfacing Lab
Dept. of EEE
Shahjalal University of Science and Technology

Project Supervisor
Md. Kabir Hasan
Lecturer,
Dept. of EEE, SUST

Contents

Abstract **3**

Introduction **5**

1 Sensor Tests **5**
1.1 Sonar Sensor . . . 5
1.1.1 Basic Understanding . . . 5
1.1.2 Circuit Diagram . . . 6
1.1.3 Uploading and Testing . . . 6
1.1.4 Code for HC-SR04 with ESP32 . . . 6
1.2 MPU6050 . . . 6
1.2.1 Basic Understanding . . . 6
1.2.2 Troubleshooting Tips . . . 7
1.2.3 How it Work . . . 7
1.2.4 Circuit Diagram and Connection . . . 7
1.2.5 Uploading and Testing . . . 8
1.2.6 Arduino Code for MPU6050 with ESP32 . . . 8
1.3 L298N Motor Driver . . . 8
1.3.1 Basic Understanding . . . 8
1.3.2 Circuit Diagram . . . 9
1.3.3 Arduino Code for L298N Using ESP32 Dev module . . . 9
1.4 Servo Motor . . . 9
1.4.1 Basic Understanding . . . 9
1.4.2 ESP-32 code for srevo motor . . . 10
1.5 Camera Module Test . . . 10
1.5.1 Basic Understanding . . . 10
1.5.2 Connections . . . 11
1.5.3 Circuit Diagram . . . 12
1.5.4 Code of ESP-32 CAM test . . . 12

2 Communication and Model Train **13**
2.1 ESP to ESP Communication . . . 13
2.1.1 Step-by-Step Communication Workflow . . . 13
2.1.2 Communication Method . . . 13
2.1.3 Code of Sender . . . 14
2.1.4 Code of Receiver . . . 14
2.2 Model Train Using Camera for Object Detection . . . 14
2.2.1 Project Overview . . . 14
2.2.2 Key Objectives . . . 14

2.2.3 Object Detection Model Training Process using Edge Impulse . . 14
2.2.4 Fire Detection with ESP32-CAM 15
2.2.5 Communication 15
2.2.6 Real-World Analogy 16
2.2.7 Summary 16

3 Design a Prototype of Swarm Robots 17
3.1 Basic Understanding 17
3.2 Forward Control Using PID Algorithm 17
3.2.1 PID-Based Forward Move Correction 18
3.3 Workflow / Methodology 18
3.3.1 Designing Master Bot 18
3.3.2 Sensor Fusion and Navigation Logic 19
3.3.3 Limitations 20
3.3.4 Circuit Diagram of Master Bot 21
3.3.5 Code of Master Bot 21
3.3.6 Designing Slave Bot 21
3.3.7 Circuit Diagram of Slave Bot 24
3.3.8 Code of Slave Bot 24

Conclusion 25

Abstract

This report presents the design and implementation of a cost-effective swarm robotics system comprising a master bot and a slave bot for autonomous fire detection and extinguishing in confined spaces. The master bot employs hybrid sensing (sonar, camera, and flame sensors) to detect and verify fires, while the slave bot navigates to the target location using ESP-NOW communication and extinguishes the fire. The system integrates edge AI for visual fire detection, sensor fusion for localization, and low-latency wireless coordination. Despite budget constraints, the prototype demonstrates robust collaboration between robots, balancing functionality and affordability. Key challenges, such as limited camera range and single-task execution, are addressed through modular hardware and prioritized workflows.

Introduction

Swarm robotics leverages decentralized coordination among simple agents to solve complex tasks, inspired by natural systems like ant colonies or bird flocks. In this project, we implement a simplified swarm model with two robots: a master bot for fire detection and a slave bot for fire extinguishing. The master bot scans its environment using a servo-mounted sonar, verifies fires via AI and flame sensors, and communicates coordinates to the slave bot via ESP-NOW. The slave bot calculates its path using trigonometry, navigates using inertial and ultrasonic data, and activates a water pump upon arrival. This system highlights principles of swarm robotics—scalability, redundancy, and distributed intelligence—while adhering to strict budget constraints. Challenges include sensor limitations, single-task execution, and communication reliability, all addressed through iterative testing and hardware-software co-design.

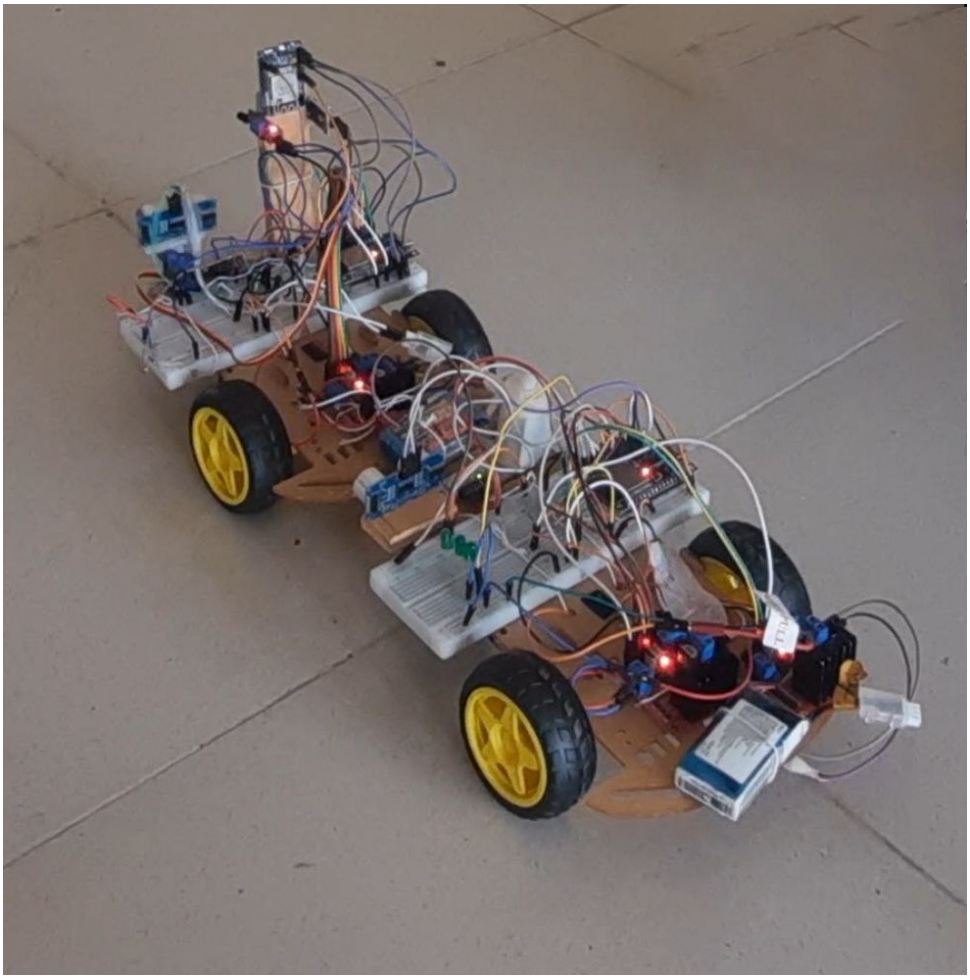

Figure 1: Implemented Swarm Robots

Chapter 1

Sensor Tests

1.1 Sonar Sensor

1.1.1 Basic Understanding

In this section, we verify the operation of an HC-SR04 ultrasonic sonar sensor connected to an ESP32 Dev Module and measure the distance of objects in front of the sensor using the ultrasonic principle.

The HC-SR04 is a low-cost ultrasonic sensor that measures distance using ultrasonic sound waves. It has four pins: $V_{\mathbf{CC}}$, **GND**, **Trig (Trigger)**, and **Echo**, where $V_{\mathbf{CC}}$ and **GND** provide power to the module.

- **Trigger (Trig):** The ESP32 sends a HIGH pulse of 10 microseconds on this pin to initiate the measurement. In this setup, we use **GPIO 13** as the trigger pin.
- **Echo:** After receiving the trigger, the sensor sends out an ultrasonic wave and sets the Echo pin HIGH. The duration of this HIGH pulse corresponds to the time taken for the ultrasonic wave to travel to the object and reflect back. We use **GPIO 12** as the echo pin.
- Using this time duration, the distance is calculated using the equation:

$$\text{Distance} = \frac{\text{Time} \times \text{Speed of Sound}}{2}$$

  This is based on the basic equation of uniform velocity: Distance = Speed × Time.

Important Note: The HC-SR04 operates at **5V**, while the ESP32's GPIO pins are designed for **3.3V** logic levels. To prevent potential damage to the ESP32:

- Use a **voltage divider** on the Echo pin, typically using two resistors such as 1kΩ and 2kΩ.
- Alternatively, you can use a **logic level shifter**.

Most ESP32 development boards provide a 5V pin, which can be used to power the HC-SR04 sensor directly.

1.1.2 Circuit Diagram

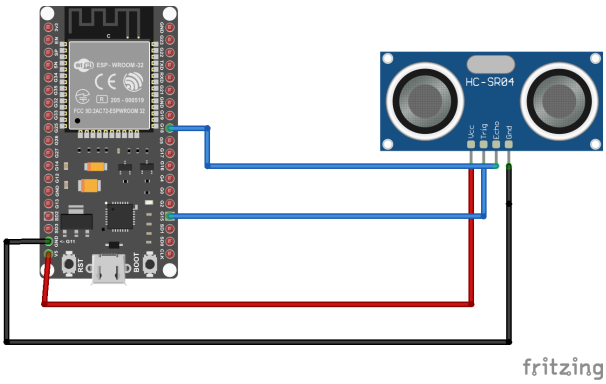

Figure 1.1: Circuit diagram of ESP32 connected with HC-SR04 ultrasonic sensor

1.1.3 Uploading and Testing

To test the setup, follow these steps:

1. Connect the ESP32 to your computer via USB.
2. open **Arduino IDE**
3. Go to **Tools** → **Board** → **ESP32 Dev Module**.
4. Select the correct **Com Port**.
5. Paste the code (provided below) into a new sketch.
6. Click Upload
7. After uploading, open the **Serial Monitor** and set the **baud rate to 115200**.
8. Place an object in front of the sensor and observe the distance values displayed.

1.1.4 Code for HC-SR04 with ESP32

Without using any built-in Library. Click → *DistanceWithoutLibrary*
Using the sonar NewPing Library. Click → *DistanceWithNewPing*

1.2 MPU6050

1.2.1 Basic Understanding

To test the MPU6050 sensor connected to the ESP32 and verify that accelerometer and gyroscope data can be successfully read and displayed using the Arduino IDE.The MPU6050 is a popular motion-tracking sensor that combines two important sensing components in a single small module:

- **Accelerometer** – Measures acceleration (movement or tilt) in 3 axes: X, Y, Z
- **Gyroscope** – Measures **rotational velocity** (how fast it's spinning) in 3 axes: X, Y, Z

1.2.2 Troubleshooting Tips

- Double-check wiring (SDA to 21, SCL to 22).
- Ensure you select the correct COM port.
- if no data appears, reset the ESP32 or check for loose wires.
- Make sure your MPU6050 is not damaged.

1.2.3 How it Work

- When you move or tilt the MPU6050, its accelerometer detects the change in speed or angle.
- If you rotate it, the gyroscope senses how fast and in what direction it's spinning
- The sensor continuously sends this data to a microcontroller (like ESP32), which can then act on it—such as stabilizing a robot or displaying orientation.

1.2.4 Circuit Diagram and Connection

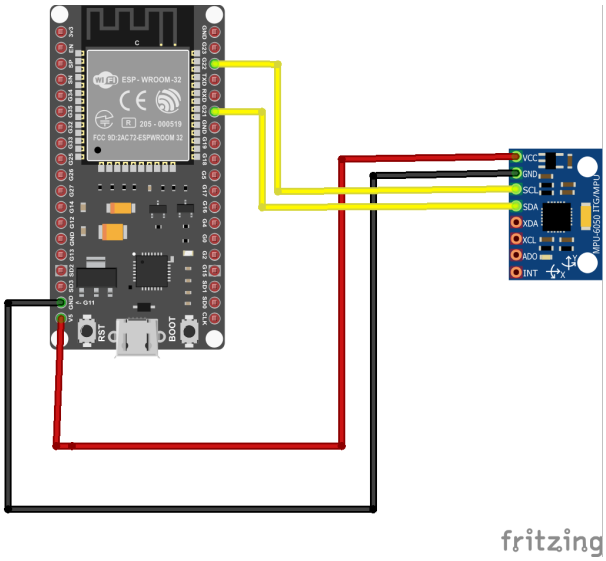

Figure 1.2: Circuit diagram of ESP32 connected with HC-SR04 ultrasonic sensor

- Connect MPU6050 $V_{\mathbf{CC}}$ and **GND** with ESP32 **5V** and **GND**.
- **SDA** → **GPIO 21**.
- **SCL** → **GPIO 22**.

1.2.5 Uploading and Testing

To test the setup, follow these steps:

1. Connect the ESP32 to your computer via USB.
2. Open **Arduino IDE**.
3. Go to **Tool** → **Board** → **ESP32 Dev Module**.
4. Select **COM Port**.
5. Upload the below Code.
6. Open **Serial Monitor**, set baud rate to **115200**.
7. Move or tilt the MPU6050 sensor and observe the real-time accelerometer and gyroscope data printed on the Serial Monitor.

1.2.6 Arduino Code for MPU6050 with ESP32

Click → *MPU6050Test*

1.3 L298N Motor Driver

1.3.1 Basic Understanding

The L298N is a popular and widely used motor driver module designed to control DC motors and stepper motors. It acts as an interface between a microcontroller (like Arduino or **ESP 32**) and motors, allowing you to control the direction and speed of motors using simple digital signals.

- Dual H-Bridge Design – Can drive 2 DC motors independently
- Wide Voltage Range (5V-35V) – Works with 6V, 12V, or 24V motors
- Peak Current: 2A per channel (sustained 1A without heatsink)
- Built-in 5V Regulator – Can power logic circuits (Arduino/ESP32)
- Speed Control – Adjust motor speed using PWM signals
- Simple Control Logic – Uses IN1, IN2, IN3, IN4 for direction

Moving Sytem

- IN1 = HIGH, IN2 = LOW → Motor A moves forward
- IN1 = LOW, IN2 = HIGH → Motor A moves backward

1.3.2 Circuit Diagram

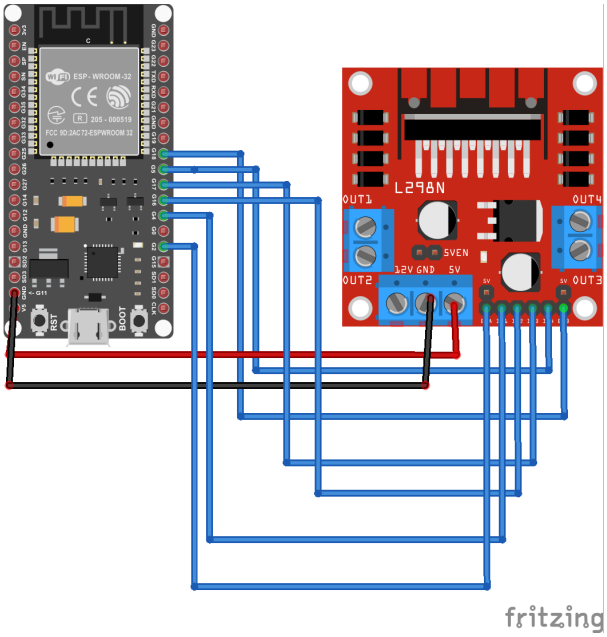

Figure 1.3: Circuit diagram of ESP32 connected with HC-SR04 ultrasonic sensor

1.3.3 Arduino Code for L298N Using ESP32 Dev module

Click → *MotorTest*

1.4 Servo Motor

1.4.1 Basic Understanding

A servo motor is a type of motor that can rotate to a specific angle based on a control signal. Unlike regular motors that spin continuously, a servo motor moves to and holds a set angle, usually between **0° and 180°**.

- A PWM signal (Pulse Width Modulation) is sent to the signal wire.
- The width of the pulse (usually between 1 ms and 2 ms) determines the angle.
- 1 ms → 0°
- 1.5 ms → 90°
- 2 ms → 180°
- The servo adjusts its position to match the signal and holds it there.

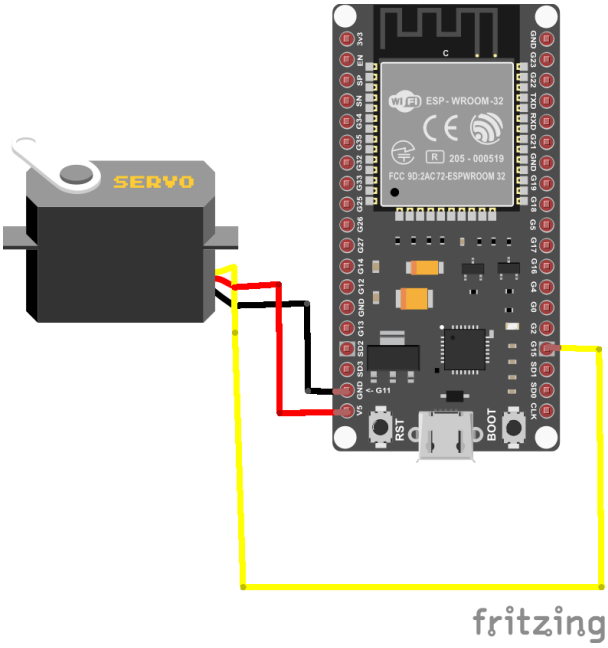

Figure 1.4: Circuit diagram of ESP32 connected with HC-SR04 ultrasonic sensor

1.4.2 ESP-32 code for srevo motor

Click → *ServoTest*

1.5 Camera Module Test

1.5.1 Basic Understanding

The ESP32-CAM is a small, low-cost microcontroller board with a built-in camera and Wi-Fi + Bluetooth capabilities. It's great for DIY projects like surveillance cameras, face detection, and home automation.

In your swarm robotics project, the ESP32-CAM can serve as the vision-enabled master robot that:

- Detects objects/fire using its camera
- Processes images with OpenCV or TensorFlow Lite
- Communicates coordinates to slave robots
- Coordinates swarm behavior

This implementation turns your ESP32-CAM into an intelligent swarm coordinator that can visually detect targets and command slave robots to respond. main we use the **ESP 32 CAM** for the long-range object detectors. we use it in the master robot **sonar sensor** but it's detecting power low so we use the camera for the long-range detection.

1.5.2 Connections

The **ESP32-CAM** is a specialized version of the ESP32 microcontroller that includes a camera module. Code is uploaded to the ESP32-CAM using another ESP32 board.

- **ESP32-CAM GND → ESP32 GND** (Common ground)
- **ESP32-CAM 5V → ESP32 VCC** (Power supply)
- **ESP32-CAM (RX) → ESP32 RX**
- **ESP32-CAM (TX) → ESP32 TX**
- **ESP32-CAM IO0 → GND (only during upload)** (Enable programming mode)

Note: ESP32-CAM requires 5V to power the camera module properly.

Programming Mode Steps

1. Connect the pins as shown in the above.
2. Connect **IO0 to GND** to enable flashing mode.
3. Plug the ESP32 module into your PC via USB.
4. In the Arduino IDE:
   - Select the correct **COM port**.
   - Select the board: `ESP32 AI THINKER Module`.
5. Click `Upload` in the Arduino IDE.
6. As it starts uploading, you may need to press the **RESET (EN)** button on the ESP32-CAM once.
7. After the upload is complete:
   - Disconnect **IO0 from GND**.
   - Press the **RESET** button again to boot normally.

1.5.3 Circuit Diagram

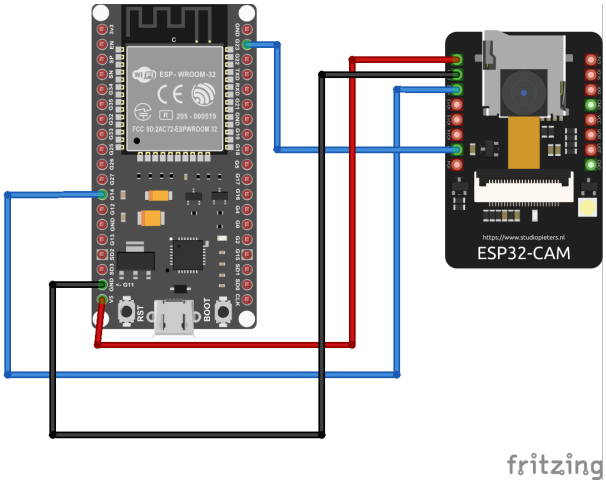

Figure 1.5: Circuit diagram of ESP32 connected with HC-SR04 ultrasonic sensor

1.5.4 Code of ESP-32 CAM test

Click → *ESP − 32CAM*

Chapter 2

Communication and Model Train

2.1 ESP to ESP Communication

2.1.1 Step-by-Step Communication Workflow

1.Code Uploading:

- First, we upload the sending code to the Sender ESP32.
- Then, we upload the receiving code to the Receiver ESP32.
- Both codes are uploaded using the Arduino IDE.

2.Operation

- The Sender ESP32 is connected to a sensor (serial monitor).
- When the sensor detects an object within a predefined area (e.g., 100 cm radius), the Sender ESP32 immediately sends a signal/message to the Receiver ESP32.

Receiver's Response

- The Receiver ESP32 receives the message.
- Upon receiving the signal, the Receiver takes necessary action to resolve the situation (e.g., stop a motor, activate an alert, or move a servo).

2.1.2 Communication Method

You can use any of the following ESP32 communication protocols

- **ESP-NOW**(Recommended for local, fast, no-router setups)
- **Wi-Fi (TCP/IP or UDP)**if you're working over a network
- **Bluetooth**for short-range communication

This structure ensures that both ESP32 boards work independently yet collaboratively, enabling smart automation and efficient wireless control in IoT or robotics projects.

2.1.3 Code of Sender

Cick → *SendingCode*

2.1.4 Code of Receiver

Click → *ReceiverCode*

2.2 Model Train Using Camera for Object Detection

2.2.1 Project Overview

This project implements a model train system equipped with a camera (ESP32-CAM) to perform object and fire detection as part of a master-slave robotic setup. The model train simulates an autonomous fire detection unit capable of identifying obstacles and fire sources along the track. Upon detection, it communicates with a slave robot to take necessary action—such as extinguishing the fire.

2.2.2 Key Objectives

- Use the **ESP32-CAM** mounted on a model train to **detect objects and fire.**
- Establish communication between a **Master (Train)** and **Slave (Extinguisher Robot).**
- Automatically navigate and stop the train when a fire or obstacle is detected.
- Trigger the slave robot to move toward Trigger the slave robot to move toward the detected area and extinguish the fire. and extinguish the fire.

2.2.3 Object Detection Model Training Process using Edge Impulse

1. **Data Acquisition**
   - Capture images (with bounding boxes) using:
     - Camera (Edge Impulse Studio / mobile app)
     - Upload from local storage
2. **Data Labeling**
   - Use Edge Impulse's bounding box editor to label objects in each image
3. **Create Impulse**
   - Define input block (Image)
   - Add processing block (e.g., Image, Resize, Grayscale or Color)
   - Add learning block (e.g., Object Detection - FOMO or YOLO)
4. **Configure Processing Block**

- Set image size (e.g., 96×96, 160×160)
- Choose color depth (grayscale or RGB)

5. **Train the Model**
   - Select model architecture (e.g., FOMO for edge devices)
   - Set training parameters (epochs, learning rate)
   - Start training on Edge Impulse cloud
6. **Evaluate Model**
   - Use validation dataset
   - Check metrics:
     - mAP (mean average precision)
     - Accuracy
     - Confusion matrix
7. **Optimize Model (Optional)**
   - Apply quantization (e.g., int8)
   - Test latency, memory footprint
8. **Model Testing**
   - Test using live camera stream or test data
   - Evaluate performance in real-world scenario
9. **Deployment**
   - Choose deployment target:
     - Arduino, ESP32, Raspberry Pi, Linux, etc.
   - Download firmware or C++ / TensorFlow Lite model
   - Flash to device

2.2.4 Fire Detection with ESP32-CAM

- **Basic:**Use image processing to detect flame color (yellow/orange-red blob detection).
- Advanced: Stream video to server (Raspberry Pi, PC) running ML-based fire detection.
- Optionally use a flame sensor for added reliability.

2.2.5 Communication

- **ESP-NOW:**Fast, no internet required. Ideal for direct ESP-to-ESP messaging.
- **Wi-Fi**Useful if operating over longer range or requiring more data.
- Slave listens for message and responds accordingly.

2.2.6 Real-World Analogy

Like a **train-mounted scout** that reports fire sightings to a **firefighter robot**. The scout doesn't act, it detects and alerts; the firefighter takes action.

2.2.7 Summary

- **Master (Train):**detects → Sends alert → Waits
- **Slave (Robot):**Receives alert → Extinguishes fire → Confirms task
- Enables efficient division of labor using camera-based object detection and wireless communication.

Chapter 3

Design a Prototype of Swarm Robots

Swarm robotics is a field of robotics where many simple robots (called agents) work together as a team to complete a task — just like how ants, bees, or birds behave in nature.

The idea is inspired by biological swarms, where individuals follow simple rules, yet the group shows intelligent and complex behavior.

3.1 Basic Understanding

The Master Bot serves as the lead unit and sensor hub in the fire-extinguishing swarm. It continuously scans the environment for potential fire sources, processes sensor data to verify a flame, and then coordinates the response by instructing the Slave Robot. In this architecture, the Master carries advanced perception hardware (a servo-mounted ultrasonic sensor, inertial measurement unit, camera, and flame sensor) and executes the decision logic. The Slave Bot, by contrast, primarily carries the extinguisher and follows the Master's commands.

The Master's role is dual:

1. Fire and identification using its sensors and AI-based vision, and
2. Coordination, in which it computes the target's position and relays this information to the Slave.

All firmware is developed under the Arduino IDE for the ESP32 microcontroller, using existing libraries for servo control, sonar, IMU, camera, and ESP-NOW communication.

3.2 Forward Control Using PID Algorithm

To ensure straightforward movement **Proportional-Integral-Derivative (PID)** control algorithm is applied. Because, the motors can't rotate at the same RPM even though the same amount of voltage and power is applied due to mechanical error and motor internal construction differences.

3.2.1 PID-Based Forward Move Correction

- **Yaw Error:** $e_{\text{yaw}}(t) = \theta_{\text{target}} - \theta_{\text{current}}(t)$
  Here, θ_{target} is the desired yaw angle (typically 0° for straight forward motion), and $\theta_{\text{current}}(t)$ is the real-time yaw angle measured by the MPU6050 sensor.

- **PID Control Signal for Orientation Correction:**

$$u(t) = K_p \cdot e_{\text{yaw}}(t) + K_i \int_0^t e_{\text{yaw}}(\tau)\, d\tau + K_d \cdot \frac{de_{\text{yaw}}(t)}{dt}$$

  where K_p, K_i, and K_d are the proportional, integral, and derivative gains respectively.

- **Motor Speed Adjustment Based on Yaw Error:**
  The control output $u(t)$ is used to compensate for the deviation in orientation:
  - If $u(t) > 0$: the robot is turning right; reduce the right motor speed or increase the left motor speed.
  - If $u(t) < 0$: the robot is turning left; reduce the left motor speed or increase the right motor speed.

  This real-time correction keeps the robot moving straight by dynamically balancing the motor speeds based on yaw feedback.

3.3 Workflow / Methodology

3.3.1 Designing Master Bot

The Master Bot's operation proceeds in a loop of sensing, analysis, and communication. The key steps are:

Environment Scanning (Ultrasonic + Servo)

A servo motor rotates the ultrasonic distance sensor through a sweep of angles. At each angle, the HC-SR04 sonar emits ultrasonic pulses and measures the round-trip echo time, giving a distance r to the nearest object in that direction. For example, rotating from 0° to 180° covers a broad frontal field. Each measurement is tagged by its servo angle θ, resulting in a set of polar coordinates (θ, r) of any detected obstacle. In practice, the HC-SR04 can sense objects roughly from 2 cm up to 4 m, with best accuracy within 0.1–2.5 m. By sweeping on a servo, the Master extends its effective view beyond a single fixed beam.

Target Locking

When the scan detects an object (potential fire) within range, the system records its polar position: the servo angle θ_1 and distance r. This defines the target's location relative to the robot's current heading. Typical logic is to flag any significant distance reading as a candidate fire location.

Alignment (MPU6050 + Chassis Rotation)

The Master then rotates its chassis to face the target angle θ_1. The onboard IMU (MPU-6050) provides real-time yaw (heading) information via its gyroscope. By integrating the gyro's angular velocity readings, the robot knows how much it has turned. For instance, if the sonar found an obstacle at $\theta_1 = 30°$ to the right, the Master will turn itself 30° clockwise, monitoring the MPU-6050 data to stop precisely at that bearing. This ensures the camera and flame sensor are pointed directly at the detected object.

Image Capture (ESP32-CAM)

Once oriented toward the target, the ESP32-CAM module takes a photo of the scene. The camera board (with a 2MP OV2640 sensor and $\sim 65°$ field of view) captures an image in which the suspected fire location is centered.

Fire Detection (AI + Flame Sensor)

The Master Bot then analyzes the camera image for signs of fire. A lightweight AI model (e.g., a small CNN or color/shape detector) runs on the ESP32 to classify flame patterns (e.g., flickering orange regions). Simultaneously, a hardware flame sensor (such as the KY-026 IR flame sensor) reads infrared light intensity. The KY-026 consists of an IR phototransistor paired with an IR LED; it outputs a digital signal when significant IR from a flame is detected. By fusing both modalities — the AI's image-based classification and the flame sensor's IR reading — the system robustly decides whether the object is indeed fire.

Communication to Slave (ESP-NOW)

If the target is confirmed to be on fire, the Master prepares a data packet containing the polar coordinates (angle θ_1 and distance r) and sends it to the Slave Bot via ESP-NOW, a low-latency peer-to-peer wireless protocol supported by ESP32 devices. ESP-NOW allows short messages (up to 250 bytes) to be exchanged without a Wi-Fi access point. Upon receipt, the Slave knows the relative bearing and distance of the fire target and can proceed to navigate toward it and activate its extinguisher.

This workflow repeats continuously: after sending a target, the Master resumes scanning for additional fires or re-evaluates if the situation changes.

3.3.2 Sensor Fusion and Navigation Logic

The Master Bot's intelligence comes from fusing its multiple sensors to accurately locate fires. Each sensor contributes complementary information:

- **Ultrasonic Sonar + Servo:** Maps obstacles around the robot by distance and bearing. However, it cannot distinguish fire from other objects.
- **Servo Angle and IMU (Orientation):** The servo's position gives the bearing θ of the sonar beam relative to the robot's chassis. Combined with yaw from the MPU-6050, the system computes the absolute bearing. For example, if $\theta_{\text{servo}} = 45°$ .we consider in our project the master bot's angle is reference angle. So , if $\theta_{\text{servo}} = 45°$

then the reference angle become 45° . This polar coordinate $(r, \theta_{\text{total}})$ localizes the object. The system can convert these to Cartesian coordinates using:

$$x = r \cdot \cos(\theta), \quad y = r \cdot \sin(\theta)$$

- **Camera (ESP32-CAM):** Captures images of suspected targets. The alignment ensures the camera centers on the potential fire. Image recognition via color/motion detection or a TinyML model verifies visual fire features.
- **Flame Sensor (KY-026):** Detects IR radiation from flames, providing a reliable close-range signal (within a few centimeters to a meter).

The navigation logic integrates these inputs: a fire is confirmed only if the camera AI or flame sensor (or both) detects it. This reduces false positives. The sonar scan provides (θ, r), which is correlated with the camera's view and the flame sensor's trigger, ensuring accurate localization.

3.3.3 Limitations

Despite its capabilities, the Master Bot has several constraints:

- **Limited Vision Range:** The ESP32-CAM's 2MP sensor with a narrow FOV limits the detection of distant or small fires. Bright or smoky conditions further reduce effectiveness.
- **Single Slave (Budget Constraint):** The system has only one Slave Bot, limiting the ability to address multiple fires simultaneously.
- **One Task at a Time:** The Master's logic handles one fire target at a time. It cannot process multiple fires concurrently.
- **Close-Range Detection:** Both the flame sensor and camera require proximity to reliably detect fire. This limits early detection capability.
- **Other Constraints:** The ultrasonic sensor may perform poorly on soft or angled surfaces and is limited by servo scanning speed. Environmental factors like smoke or light can degrade sensor accuracy.

These limitations should be considered in system design. Adding more slaves or upgrading sensors could enhance performance, but within the one-master, one-Slave framework, these define the performance envelope.

3.3.4 Circuit Diagram of Master Bot

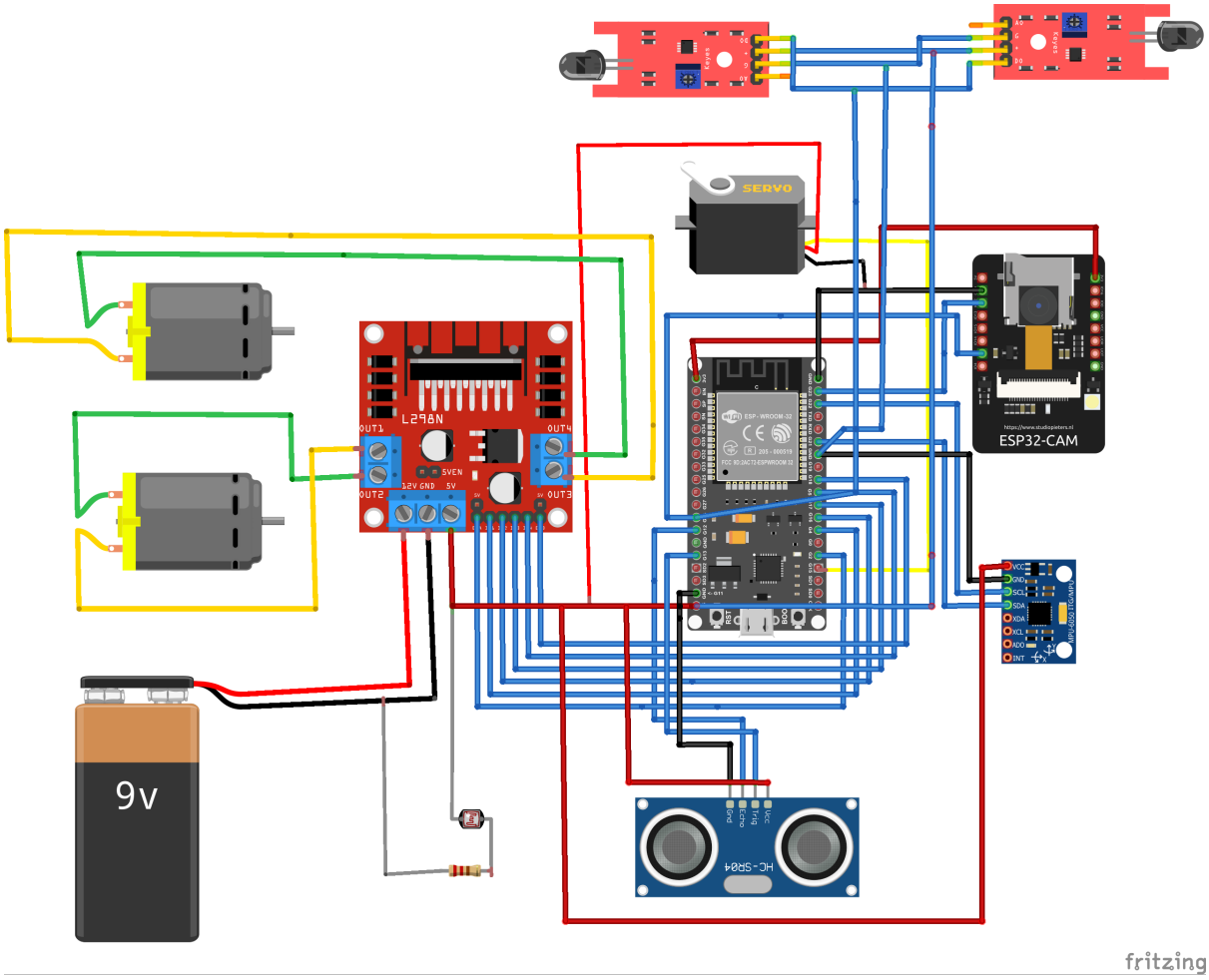

Figure 3.1: Master bot circuit diagram

3.3.5 Code of Master Bot

Click → *MasterBot*

3.3.6 Designing Slave Bot

The Slave Bot is a mobile firefighting unit that receives target coordinates from the Master Bot and autonomously navigates to extinguish the fire. Its hardware consists of an ESP32 microcontroller, an ultrasonic distance sensor, an MPU6050 inertial measurement unit (IMU), an L298N dual H-bridge motor driver, and a small water pump. The ESP32 serves as the central controller, handling communication and sensor data processing. The MPU6050 (a 3-axis accelerometer and gyroscope) provides orientation allowing the robot to measure its rotation. The ultrasonic sensor (such as an HC-SR04) measures distance by emitting an ultrasonic pulse and timing the echo. This sonar is used to detect obstacles or determine when the robot is close to the fire. The L298N driver powers the DC drive motors and the water pump, enabling controlled motion and actuation. All components are powered by onboard batteries, making the Slave Bot self-contained and mobile.

Communication with Master

The Slave Bot communicates with the Master Bot using ESP-NOW, a wireless protocol for ESP32 devices. ESP-NOW enables direct, low-latency peer-to-peer communication without a route. In our system, the Master broadcasts the fire location (angle θ_1 and distance r) to the Slave by MAC-address pairing. ESP-NOW supports one-to-many connections so the Master and Slave can be paired as a one-to-one link. This channel provides millisecond-level response times and long-range coverage which is ideal for timely firefighting commands. Upon power-up, the Slave enters a listening state, waiting for the Master's coordinate message.

Workflow and Navigation

Upon receiving the fire coordinates from the Master, the Slave Bot computes its own path. Denote the Master–Slave separation distance by x (fixed by their formation). The fire's coordinates relative to the Master are given in polar form (r, θ_1). The Slave transforms these into its own reference frame. For example, the required heading θ_2 (relative to the Slave's forward direction) can be calculated by planar geometry, e.g.

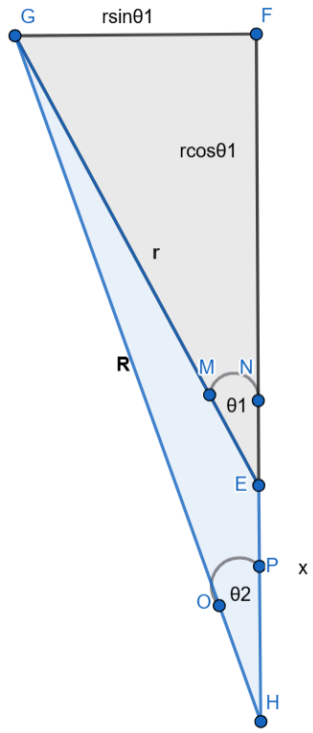

Figure 3.2: Position Finding ($\theta_1 > 90°$)

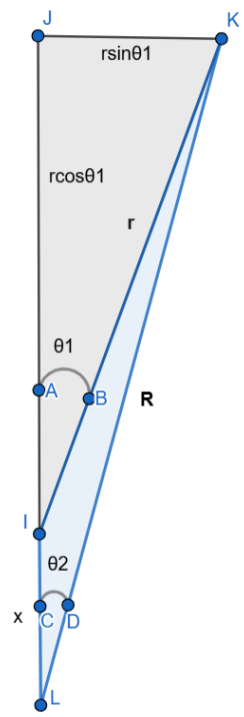

Figure 3.3: Position Finding ($\theta_1 < 90°$)

$$\theta_2 = \begin{cases} \tan^{-1}\left(\dfrac{r\sin\theta_1}{r\cos\theta_1 + x}\right), & \text{if } \theta_1 < 90° \\[2ex] \tan^{-1}\left(\dfrac{r\sin\left(\theta_1 - \frac{\pi}{2}\right)}{r\cos\left(\theta_1 - \frac{\pi}{2}\right) + x}\right) + \dfrac{\pi}{2}, & \text{if } \theta_1 \geq 90° \end{cases} \tag{1}$$

The bot then performs an in-place rotation to align with angle θ_2. This turn is monitored by the MPU6050: the gyroscope output provides real-time yaw feedback, enabling precise rotation to the desired heading:contentReference[oaicite:6]index=6. Once correctly oriented, the Slave drives forward toward the target.

During travel, the Slave continuously reads the ultrasonic sensor to detect obstacles or the target. When the sonar reports a distance below the trigger threshold (approximately 5–10 cm), it indicates that the robot is directly at the fire or an obstruction. The Slave then stops its forward motion. Next, it activates the water pump through the L298N driver to spray water on the fire. The pump runs for a preset duration or until the fire is deemed extinguished. After extinguishing (or if the water supply is depleted), the Slave stops the pump and returns to an idle state, ready for the next task.

Sensor Fusion

The Slave Bot integrates data from multiple sensors to navigate reliably. Inertial data from the MPU6050 (accelerometer and gyro) are fused with distance readings from the ultrasonic sonar. For example, the gyroscope ensures accurate heading control during rotation, while the accelerometer can detect any tilt or vibration. The sonar provides distance-to-target information. In advanced systems, such multi-sensor fusion is often implemented via Kalman filtering or similar algorithms which combine and smooth measurements to improve accuracy. In our prototype, a simple approach is used: the MPU6050 governs rotation accuracy and stability, and the ultrasonic sensor enforces a safe stopping distance. Nevertheless, the principle is that combining inertial and range data yields more robust pose estimation and collision avoidance than either sensor alone.

Limitations

This slave-unit design has several constraints. It is limited to handling one fire at a time and depends entirely on the Master for fire detection (it has no independent flame sensor). Its navigation is very basic: the Slave drives in a straight line to the target and has no map or obstacle avoidance beyond the frontal sonar. The fire must lie within a short range since the ultrasonic sensor must approach within about 70-80 cm to trigger extinguishing. Ultrasonic sensing itself is imprecise – the sonar's wide beam and reflections can cause noisy distance measurements and environmental factors (noise, temperature) can degrade accuracy. The system also includes only a single Slave Bot, so it cannot tackle multiple fires in parallel. Finally, with low-cost hardware and simple control algorithms, the Slave's movements are not optimized: it stops immediately at the obstacle threshold and does not attempt complex maneuvers. These limitations reflect the prototype nature of the project but would need to be addressed in a full-scale firefighting deployment.

3.3.7 Circuit Diagram of Slave Bot

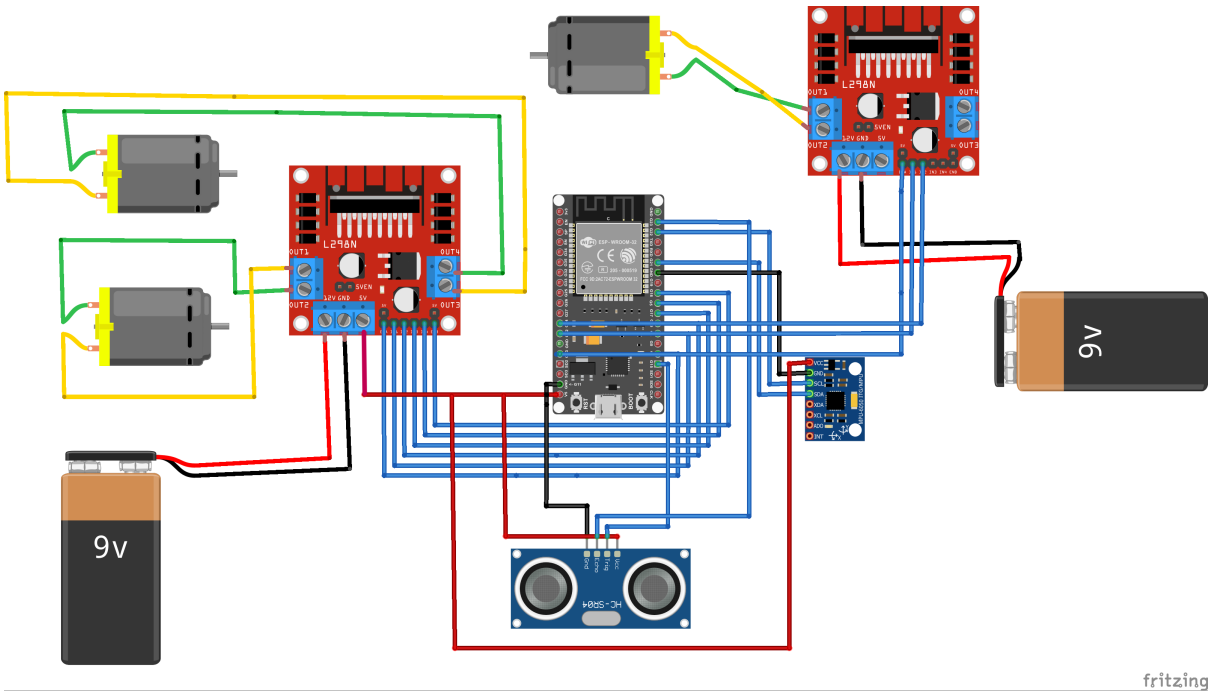

Figure 3.4: Slave bot circuit diagram

3.3.8 Code of Slave Bot

Click → *SlaveBot*

Conclusion

This project successfully demonstrates a collaborative robotics system for fire emergencies using a master-slave architecture. The master bot's hybrid sensing (sonar, camera, and flame sensors) ensures accurate fire detection, while the slave bot's navigation and extinguishing mechanisms validate the practicality of swarm principles. Key achievements include:
1. Sensor Fusion: Combining ultrasonic, inertial, and visual data for reliable target localization.
2. Edge AI: Implementing lightweight fire detection on the ESP32-CAM.
3. ESP-NOW Coordination: Achieving low-latency communication without external networks.

Limitations such as limited camera range and single-slave scalability highlight areas for future improvement, including multi-agent coordination and thermal imaging. Despite these constraints, the system provides a foundation for affordable, autonomous fire-response solutions in small-scale environments.

All the codes are given in this link below:
https://github.com/rashed-23/Swarm_Robots_and_Fire_Fighting.git

You can also find all the codes given in this link below:
https://github.com/saeed-5340/Swarm_Robotics_Concept_Porototype